



Enseirb-Matmeca
Option Robotique et Apprentissage

Etat de l'art Projet Robotique

travail réalisé par

César LARRAGUETA, Olivier ROUAULT, Antony THIERY

Etat de l'Art : Robot Socker Kit

Année universitaire 2025 - 2026

Encadrement

Titre Marc DUCLUSAU
Titre Vincent PADOIS
Titre Grégoire PASSAULT

Client
Encadrant
Client

Résumé

Cet état de l'art se concentre sur l'amélioration des simulateurs pour les robots mobiles, en particulier la plateforme Robot Soccer Kit (RSK), ainsi que sur les techniques pour réduire l'écart entre simulation et réalité (sim-to-real gap). Le problème du simulateur actuel de RSK réside dans l'utilisation d'un modèle simplifié des robots, qui ne prend pas en compte les frottements, les glissements, les dynamiques des moteurs ou les bruits des capteurs, ce qui limite sa fidélité.

En effet, le modèle actuel se base sur la cinématique des robots omnidirectionnels, décrite par une pose et une vitesse de châssis. Le simulateur utilise une matrice de cinématique inverse pour convertir les vitesses en commandes de roues.

Il existe cependant des simulateurs qui corrigent les écarts de simulation en utilisant des réseaux de neurones pour prédire les états réels à partir des états simulés. En ce qui concerne les interactions entre objets, certains réseaux décomposent les systèmes en objets et relations, permettant une généralisation à des configurations variées. Il est ainsi possible de modéliser les déplacements de la balle suite aux interactions avec les robots.

Table des matières

Table des figures	vii
Introduction	1
1 Modèles analytiques et simulation de robots omnidirectionnels	3
1.1 Cinématique des robots omnidirectionnels	3
1.2 Prise en compte de la dynamique et des effets non idéaux	4
1.3 Limites de ces modèles dans le contexte de RSK	6
2 Modèles appris et simulateurs neuronaux pour la dynamique robotique	9
2.1 Augmentation de la simulation par calibration du modèle	9
2.2 Augmentation de la simulation par correction d'écarts entre simulation et réalité	10
2.3 Intégration des interaction entre objet en simulation	11
3 Apprentissage par renforcement et Sim-to-real pour le robot soccer	13
3.1 Apprentissage par renforcement en robotique	13
3.2 RL pour le <i>Robot Soccer</i>	20
3.3 Techniques de réduction de l'écart sim-to-real	22
Conclusion & perspectives	25
Bibliographie	28

Table des figures

3.1	Tirée de Kober <i>et al.</i> (2013) [6]. (a) Le robot Obelix est un robot roulant ayant appris à pousser des objets. (b) Le bras robotique Zebra Zero a appris à mettre une cheville dans un trou. (c) Un hélicoptère a appris un contrôleur de vol. (d) Le robot humanoïde DB de l'entreprise Sarcos utilise des modèles prédictifs pour équilibrer une perche.	14
3.2	La Figure 1 de l'article de Tirumala et al. présente l'environnement d'entraînement. Les agents sont appris entièrement en simulation, dans un environnement aligné avec celui du monde réel afin de permettre un transfert direct sans réentraînement. En simulation, la vue de la caméra est générée à l'aide d'un rendu NeRF du terrain et de l'arrière-plan statiques, auquel on ajoute les objets dynamiques.	21
3.3	La Figure 3 de l'article de Tirumala et al. illustre plusieurs comportements émergents appris par l'agent.	21

Introduction

La plateforme Robot Soccer Kit (RSK) est une plateforme pédagogique de robotique mobile conçue pour l'apprentissage de la robotique et de l'intelligence artificielle [2, 10]. Elle met en scène de petits robots à roues holonomes évoluant sur un terrain de football miniature, dont la position est suivie par une caméra fixe placée au-dessus du terrain, puis transmise à un logiciel central [10]. Les utilisateurs commandent les robots en leur envoyant des consignes de vitesse de châssis via une API simple, sans avoir à gérer la localisation ni la communication bas niveau [2]. RSK constitue à la fois un outil d'enseignement et un support de compétition entre étudiants ou passionnés [10].

Dans ce contexte, la question de la simulation physique des robots occupe une place centrale. Un simulateur de base est déjà fourni avec la plateforme, mais il repose sur un modèle analytique relativement simple [2]. La cinématique omnidirectionnelle du châssis y est correctement prise en compte, en s'appuyant sur des modèles classiques de bases mobiles holonomes [8], tandis que la dynamique du robot, les frottements, le glissement des roues, les limitations des moteurs ou encore le bruit de mesure ne font l'objet que de modèles grossiers [2]. Il en résulte un écart significatif entre les trajectoires simulées et les trajectoires observées sur le robot réel, écart qui rejoint les difficultés plus générales de transfert simulation-réalité décrites dans la littérature sur la robotique mobile [17]. Cet écart limite la confiance accordée à la simulation, en particulier pour la conception de lois de commande avancées ou l'entraînement de politiques d'apprentissage par renforcement destinées à être transférées sur la plateforme physique [6, 17].

De nombreux travaux ont déjà étudié séparément différents aspects de ce problème. La littérature propose des modèles cinématiques analytiques pour les robots mobiles omnidirectionnels [8], des approches d'identification et de calibration de modèles de mouvement à partir de données [4, 7], ainsi que des méthodes d'apprentissage par renforcement appliquées à des robots mobiles ou à des scénarios proches du robot soccer [11, 15, 16]. La littérature décrit également des simulateurs hybrides ou augmentés, qui corrigent ou complètent un modèle physique existant à l'aide de composantes neuronales [1, 3, 5, 9, 14]. Cependant, ces travaux ciblent souvent d'autres plateformes, des robots plus coûteux ou des environnements de simulation plus lourds que ceux visés par RSK.

Ces approches présentent donc plusieurs limites lorsqu'il s'agit de les transposer directement au cas de RSK. D'une part, les modèles analytiques classiques reposent sur des hypothèses idéalisées concernant la dynamique et l'adhérence au sol, qui ne correspondent pas toujours au comportement observé sur de petits robots holonomes comme ceux de RSK [4, 7, 8]. D'autre part, de nombreuses méthodes de modélisation ou de simulation apprises s'appuient sur des volumes de données importants, des capacités de calcul élevées ou des infrastructures logicielles complexes [1, 5, 9, 14], dont le déploiement dans un cadre pédagogique reste difficile. Enfin, les travaux en apprentissage par renforcement pour le robot soccer s'appuient fréquemment sur des simulateurs déjà très détaillés [11, 15, 16], alors que le simulateur actuellement disponible pour RSK demeure

relativement simple sur plusieurs aspects physiques (frottements, glissement, bruit de mesure) [2, 10], ce qui conduit à un écart simulation–réalité plus marqué.

Dans ce contexte, ce rapport propose un état de l’art structuré autour de ces enjeux, en prenant la plateforme RSK comme fil conducteur. Le premier apport consiste à analyser, à partir de la littérature et du code existant, les modèles analytiques et les choix de simulation utilisés pour les robots omnidirectionnels et, en particulier, à situer le simulateur de RSK par rapport à ces approches, en mettant en évidence leurs hypothèses et leurs limites du point de vue du transfert simulation–réalité [2, 8, 10]. Le deuxième apport consiste à synthétiser les travaux récents sur les modèles appris et les simulateurs neuronaux pour la dynamique robotique, afin d’identifier les idées susceptibles d’être adaptées à RSK tout en restant compatibles avec ses objectifs pédagogiques [1, 3, 4, 5, 7, 9, 14]. Le troisième apport consiste à situer les méthodes d’apprentissage par renforcement et les techniques de réduction de l’écart sim–to–real dans le contexte du robot soccer, afin de dégager des pistes d’intégration future d’une composante intelligence artificielle sur la plateforme [6, 11, 15, 16, 17].

Le rapport est structuré en trois chapitres principaux. Le chapitre 1 présente les modèles analytiques de robots omnidirectionnels et les approches classiques de simulation, en les mettant en regard de la cinématique et du simulateur utilisés dans RSK, afin de montrer comment la plateforme s’inscrit dans ce cadre et quelles limites apparaissent dans une perspective sim-to-real. Le chapitre 2 est consacré aux modèles appris et aux simulateurs neuronaux pour la dynamique robotique, en abordant successivement la calibration de modèles, la correction des écarts entre simulation et réalité, la prise en compte des interactions entre objets et les critères permettant d’évaluer la qualité d’un simulateur appris. Le chapitre 3 traite de l’apprentissage par renforcement et des approches sim-to-real dans le contexte du robot soccer, et discute leur application possible à la plateforme RSK. Enfin, la conclusion synthétise les principaux enseignements de cet état de l’art et propose des pistes de travaux futurs pour le développement d’outils de simulation et de méthodes d’IA sur la plateforme.

1 Modèles analytiques et simulation de robots omnidirectionnels

Ce chapitre s'intéresse aux modèles analytiques utilisés pour décrire et simuler le mouvement des robots omnidirectionnels de manière générale, en prenant la plateforme Robot Soccer Kit (RSK) comme exemple représentatif. Il présente d'abord la cinématique des bases holonomes et la façon dont la relation entre vitesse de châssis et vitesses de roues est formalisée, puis montre comment ce cadre est implémenté dans le simulateur RSK. Il décrit ensuite la dynamique simplifiée introduite dans la simulation, qui agit directement au niveau des vitesses et des collisions. Enfin, il discute les limites de ce modèle au regard des effets non idéaux observés sur les robots réels et des travaux de calibration proposés dans la littérature, ce qui prépare les approches basées sur des modèles appris présentées au chapitre suivant.

1.1 Cinématique des robots omnidirectionnels

Un robot omnidirectionnel RSK se décrit par sa pose dans le plan du terrain

$$q = (x, y, \theta)$$

où x, y sont les coordonnées du centre du robot dans le repère du terrain, et θ son orientation. Cette écriture dans le code de RSK est compatible avec le formalisme utilisé dans *ModernRobotics*, où la configuration d'un robot mobile est notée $q = (\phi, x, y)$, avec ϕ l'orientation du robot par rapport au repère du terrain [8]. Dans le cadre du simulateur RSK, la commande n'agit pas directement sur la position q , mais sur une vitesse de châssis exprimée dans le repère du robot. Cette vitesse est notée

$$s = (v_x^{robot}, v_y^{robot}, \omega),$$

où v_x^{robot} désigne la vitesse vers l'avant du robot, v_y^{robot} la vitesse latérale, et ω la vitesse de rotation autour de l'axe vertical. Cette quantité correspond au vecteur vitesse (*twist*) V^b du châssis dans le formalisme de *ModernRobotics*[8].

Pour un robot à trois roues omnidirectionnelles, la cinématique instantanée impose une relation linéaire entre la vitesse du châssis s et les vitesses des roues $w = (\omega_1, \omega_2, \omega_3)$. À partir de la géométrie du robot (angles d'orientation des roues et distance entre chaque roue et le centre du châssis), il est possible de construire une matrice de cinématique inverse IK telle que

$$w = IKs.$$

Cette matrice IK , implémentée dans le code RSK, est obtenue à partir des vecteurs de direction de chaque roue et du rayon des roues, conformément à la démarche présentée dans *ModernRobotics*[8]. Elle formalise la question cinématique centrale des robots

omnidirectionnels : pour une vitesse de châssis donnée, quelles vitesses doivent être appliquées aux roues. Dans le simulateur RSK, la relation châssis–roues décrite par IK est utilisée principalement pour vérifier que la vitesse demandée s respecte les contraintes physiques des actionneurs, en particulier une vitesse maximale par roue. Une fois cette saturation effectuée, les vitesses de roues ne sont plus manipulées explicitement et la suite du calcul se fait au niveau du châssis. La cinématique du mouvement dans le repère du terrain est alors obtenue en projetant la vitesse linéaire du châssis dans le repère monde à l’aide de la matrice de rotation $R(\theta)$ associée à l’orientation du robot, puis en intégrant cette vitesse afin de mettre à jour la pose q . Les choix d’intégration numérique et la prise en compte, de manière simplifiée, de la dynamique seront détaillés dans les sections suivantes.

1.2 Prise en compte de la dynamique et des effets non idéaux

La cinématique décrite précédemment relie la vitesse du châssis à l’évolution de la pose, sans se soucier des forces et des couples qui produisent ce mouvement. En pratique, entre la consigne de vitesse envoyée au robot et le mouvement réellement observé interviennent de nombreux phénomènes dynamiques : inertie du châssis, limitations des moteurs, frottements et glissements de roues, délais dans la chaîne de commande, bruit et latence des capteurs de localisation. Le simulateur RSK ne cherche pas à modéliser ces effets de manière détaillée, mais introduit une dynamique simplifiée qui agit directement au niveau des vitesses. Chaque robot simulé est associé à une consigne de vitesse de châssis $s = (v_x^{robot}, v_y^{robot}, \omega)$ exprimée dans le repère du robot. Lorsque cette consigne est mise à jour, le simulateur commence par vérifier qu’elle respecte les limites physiques simples imposées au niveau des roues. La cinématique inverse est utilisée pour calculer les vitesses de roues $w = IKs$ correspondant à la consigne. Si l’une de ces vitesses dépasse la vitesse maximale autorisée pour une roue, un facteur d’échelle est calculé de façon à ramener toutes les vitesses de roues en dessous de cette limite. La même échelle est ensuite appliquée à la consigne de vitesse du châssis, ce qui garantit, de manière purement cinématique, que les moteurs ne sont pas sollicités au-delà de leur vitesse maximale tout en conservant la direction de la commande. La dynamique intervient ensuite dans la manière dont la vitesse réelle du robot se rapproche de cette consigne saturée. À chaque pas de simulation de durée Δt , le simulateur dispose de la vitesse actuelle v du robot et de la vitesse cible v^* . Plutôt que d’appliquer instantanément v^* , une fonction de limitation de variation décrit une nouvelle vitesse $v_{nouvelle}$ telle que la norme de $v_{nouvelle} - v$ ne dépasse pas une variation maximale prescrite. En d’autres termes, la variation de vitesse est bornée, ce qui revient à imposer une accélération maximale en norme, différente pour la partie linéaire et pour la partie angulaire. Cette règle donne une dynamique de premier ordre sur la vitesse du châssis : le robot ne peut pas changer brutalement de vitesse, mais la masse, le moment d’inertie ou les courbes couple–vitesse des moteurs n’apparaissent jamais explicitement.

Une fois la vitesse du châssis mise à jour, la composante linéaire est exprimée dans le repère du terrain par rotation en fonction de l’orientation θ du robot. La pose $q = (x, y, \theta)$ de chaque objet simulé (robots et balle) est ensuite intégrée par un schéma discret très

simple. Si l'on note $(v_x^{monde}, v_y^{monde})$ la vitesse linéaire dans le repère du terrain et ω la vitesse angulaire, la mise à jour s'écrit

$$x_{k+1} = x_k + v_{x,k}^{monde} \cdot dt, y_{k+1} = y_k + v_{y,k}^{monde} \cdot dt, \theta_{k+1} = \theta_k + \omega_k \cdot dt.$$

Les interactions entre objets sont modélisées par un schéma de collision élastique entre disques de rayon donné. Pour deux objets en contact, le simulateur construit d'abord un repère local de collision. Le vecteur normal n est défini comme le vecteur unitaire allant du centre du premier objet vers le second, et un vecteur tangent t est obtenu en faisant tourner n de 90 degrés dans le plan. Dans ce repère (n, t) , la vitesse de chaque objet est décomposée en composantes normale et tangentielle. Le modèle de collision n'agit que sur la composante normale : pour des masses m_1 et m_2 , des vitesses normales incidentes u_1 et u_2 et un coefficient de restitution fixé à $C_r = 0.25$, ce qui correspond à un choc fortement amorti, les nouvelles vitesses normales après choc sont données par les formules de collision unidimensionnelle suivantes,

$$u'_1 = \frac{m_1 u_1 + m_2 u_2 + m_2 C_r (u_2 - u_1)}{m_1 + m_2}, u'_2 = \frac{m_1 u_1 + m_2 u_2 + m_1 C_r (u_1 - u_2)}{m_1 + m_2}$$

Les composantes tangentielles sont conservées, puis les vitesses complètes sont reconstruites dans le repère du terrain en remultipliant par la matrice de passage. Dans une seconde étape, avant de valider la nouvelle position, le simulateur annule toute composante de vitesse normale qui conduirait à une interpenetration persistante, de manière à maintenir les disques juste au contact ou en séparation. La balle subit en outre une décélération constante, appliquée à sa vitesse linéaire, qui modélise les pertes d'énergie dues aux frottements au sol. Lorsqu'elle sort du rectangle correspondant au tapis de jeu modélisé, elle est simplement réinitialisée au centre du terrain avec une vitesse nulle, ce qui constitue une règle de remise en jeu simplifiée propre au simulateur RSK.

Ce traitement donne un modèle dynamique cohérent avec l'objectif de simulation légère : les vitesses sont saturées et lissées dans le temps, la pose est intégrée de façon explicite, et les collisions sont gérées par un modèle de choc simple au niveau des vitesses. En contrepartie, de nombreux phénomènes physiques réels, tels que la dynamique détaillée des moteurs, les frottements continus des roues ou le bruit de mesure des capteurs, ne sont pas représentés. [2]

Dans le contexte des robots mobiles, plusieurs travaux analysent explicitement l'impact des effets dynamiques non idéaux sur l'odométrie et la localisation. Lemmer et al. proposent de distinguer deux familles d'erreurs dans le modèle d'odométrie : des erreurs systématiques, liées aux paramètres géométriques et physiques du robot (diamètre effectif des roues, entraxe, tolérances de montage, calibration des capteurs), et des erreurs dynamiques ou non systématiques, associées à l'interaction avec l'environnement (irrégularités du sol, glissements de roues, variation du diamètre des pneus en charge, etc.) [7]. Les erreurs systématiques sont considérées comme approximativement constantes sur un cycle d'utilisation et peuvent être estimées, alors que les erreurs dynamiques ne peuvent pas être supprimées complètement. Pour réduire l'erreur d'odométrie, les auteurs s'appuient sur un système de localisation externe de haute précision, qui fournit une trajectoire de référence. Cette trajectoire est utilisée pour ajuster finement les paramètres des capteurs et des actionneurs, de manière à rapprocher le comportement du modèle analytique des déplacements effectivement observés.

Fazekas et al. étudient un problème similaire dans le contexte d'un véhicule routier, où la dynamique est plus non linéaire, notamment à cause des angles de dérive latérale et des effets de contact entre pneus et chaussée[4]. Leur modèle d'odométrie reste fondé sur une description physique détaillée du véhicule, mais son identification est compliquée par le bruit des capteurs et par la propagation de ce bruit au travers des non-linéarités du modèle. Les auteurs défendent l'idée que la priorité doit rester à une modélisation physique soignée et à la calibration des paramètres sur des données expérimentales, les méthodes d'apprentissage n'intervenant qu'en complément. Ils proposent pour cela un schéma d'optimisation de type Gauss-Newton appliqué à des segments de trajectoire, dans lequel chaque segment est pondéré par un poids appris. Cette pondération permet de diminuer l'influence des portions de trajectoire les plus perturbées par les frottements, les glissements ou le bruit de mesure, tout en conservant la structure analytique du modèle dynamique.

Ces deux études illustrent une manière de gérer les effets non idéaux qui reste entièrement fondée sur un modèle dynamique analytique, mais dont les paramètres sont systématiquement ajustés à partir de trajectoires mesurées. Dans ce cadre, la cinématique et la structure des équations d'odométrie sont conservées, mais la calibration joue un rôle central pour rapprocher le modèle du comportement réel. Pour un simulateur léger comme celui du Robot Soccer Kit, ces travaux ne définissent pas une référence à reproduire, mais indiquent le type de démarches qui pourraient être envisagées si l'objectif était de réduire l'écart entre la dynamique simulée et celle observée sur les robots physiques. Les limites du modèle actuellement implémenté dans RSK seront discutées plus en détail dans la section suivante.

1.3 Limites de ces modèles dans le contexte de RSK

Au vu des modélisations présentées dans les sections précédentes, le simulateur RSK repose sur un modèle dynamique très simplifié : la commande agit au niveau de la vitesse du châssis, cette vitesse est simplement limitée en variation, la pose (x, y, θ) est intégrée par un schéma explicite de type Euler, les collisions sont représentées par un choc élastique unidimensionnel, et la pose des robots est supposée connue sans bruit ni retard. Ce modèle est cohérent avec l'objectif de disposer d'un simulateur léger, mais il ne constitue qu'une première approximation de la dynamique réelle, où interviennent glissement continu des roues, dynamique des moteurs et incertitudes de localisation.

Une première limite est l'absence explicite de loi de suivi de trajectoire dans la boucle de simulation. Le simulateur reçoit, à chaque pas de temps, une consigne de vitesse du châssis, sans décrire comment cette consigne est obtenue à partir d'une trajectoire de référence. *ModernRobotics* propose la modélisation de commande suivante, en considérant une trajectoire voulue $q_d(t)$, décrivant la pose de référence en fonction du temps, il est possible de définir une vitesse à appliquer de la forme

$$\dot{q}(t) = \dot{q}_d(t) + K_p(q_d(t) - q(t)) + K_i \int_0^t (q_d(\tau) - q(\tau))d\tau.$$

Ici, $q(t)$ désigne la pose mesurée, $\dot{q}(t)$ la vitesse de référence, K_p et K_i des gains proportionnel et intégral. Le terme proportionnel corrige l'erreur instantanée de pose, le

terme intégral compense des biais persistants. Une telle loi, du type de celles décrites dans *ModernRobotics* pour les bases mobiles, permet de relier directement trajectoire et consigne de vitesse ; son intégration dans RSK serait une première piste pour rapprocher trajectoires simulées et trajectoires réellement exécutées.[8]

Une autre question concerne la calibration des paramètres du modèle. Dans les travaux de calibration d'odométrie (par exemple Lemmer et al. et Fazekas et al.), la structure analytique des équations est conservée, mais les paramètres géométriques et dynamiques (rayon effectif des roues, entraxe, coefficients liés aux dérives et aux frottements, etc.) sont identifiés à partir de trajectoires mesurées par un système de référence précis. Ces études montrent qu'un modèle analytique dont les paramètres n'ont pas été ajustés sur des données réelles laisse subsister des erreurs significatives dès que les effets non idéaux deviennent importants. Dans la documentation disponible sur le Robot Soccer Kit, aucune procédure de calibration systématique n'est décrite pour le simulateur. Une piste d'évolution naturelle consisterait à comparer des trajectoires réelles et simulées, puis à ajuster certains paramètres (par exemple un rayon de roue effectif, des accélérations maximales ou des paramètres de collision) afin de réduire l'écart entre dynamique simulée et dynamique observée.

Un modèle analytique simplifié, comme celui utilisé dans le simulateur RSK, ne constitue qu'une première approximation de la dynamique réelle. Les travaux de calibration montrent qu'il est nécessaire d'ajuster précisément les paramètres du modèle à partir de trajectoires mesurées et, si besoin, d'enrichir la modélisation pour mieux représenter frottements, glissements et bruit [4, 7]. Par ailleurs, des travaux plus récents proposent de compléter ces modèles physiques par des composantes neuronales chargées de corriger directement les résidus d'erreur entre simulation et réalité [9, 14]. Dans cette perspective, le simulateur RSK peut être vu comme un noyau analytique à calibrer et à éventuellement augmenter par des modules appris, en particulier dans une optique de sim-to-real.

2 Modèles appris et simulateurs neuronaux pour la dynamique robotique

Les simulateurs dynamiques classiques, bien qu'indispensables pour le développement et le test des systèmes robotiques, souffrent de limitations majeures en raison des imperfections des modèles physiques. Ces imperfections, telles que les incertitudes sur les paramètres mécaniques, les frottements non modélisés, les jeux mécaniques, ou encore les dynamiques complexes des actionneurs, entraînent un écart significatif entre les comportements simulés et réels, communément appelé "sim to real gap". Cet écart limite la fiabilité des simulations pour des applications critiques, comme le contrôle en boucle fermée ou la planification de trajectoires.

Pour pallier ces limitations, une approche émergente consiste à augmenter les simulateurs existants avec des modèles neuronaux, plutôt que de remplacer entièrement les simulateurs physiques par des réseaux de neurones. Cette hybridation permet de conserver les avantages des simulateurs physiques (interprétabilité, respect des lois de la physique) tout en corrigeant leurs imperfections grâce à l'apprentissage automatique.

2.1 Augmentation de la simulation par calibration du modèle

Le modèle des robots RSK ainsi que les caractéristiques du terrain comportent inévitablement des imperfections. Celles-ci entraînent une dérive progressive entre la position estimée et la position réelle : pour un même ordre envoyé aux actionneurs, le déplacement obtenu varie selon la position initiale du robot, l'état des roues, les frottements, l'usure, les glissements, les défauts mécaniques ou encore la dérive des capteurs. Le simulateur, fondé sur un modèle idéalisé qui ne prend pas en compte l'ensemble de ces effets, amplifie cette divergence au fil des commandes.

Pour limiter cette dérive, plusieurs approches de calibration exploitent des données issues des capteurs internes et externes, ainsi que des trajectoires prédéfinies, c'est le cas dans cette étude de Lemmer sur la calibration de rover miniature (2010) [7]. Afin d'améliorer la fiabilité des estimations et de réduire l'influence du bruit, ces méthodes segmentent généralement les données de déplacement en petites portions. Chaque segment permet alors de recalculer les paramètres du modèle sur un intervalle court, ce qui atténue l'impact des erreurs locales ou des perturbations ponctuelles.

La démarche repose ensuite sur un modèle mathématique où les constantes sensibles à l'environnement ou susceptibles d'évoluer avec le temps sont considérées comme ajustables. Des techniques d'apprentissage, telles que les réseaux de neurones multicouches, peuvent être employées pour déterminer automatiquement les paramètres les plus cohé-

rents avec le comportement réel. C'est précisément l'approche mise en avant dans l'étude de Fazekas et Gáspár (2024) [4], qui propose de pondérer intelligemment ces segments grâce à un réseau de neurones chargé d'identifier ceux dont les données sont fiables et ceux affectés par du bruit ou des anomalies. Cette pondération est ensuite intégrée dans une calibration globale, ce qui renforce la robustesse du modèle et permet de compenser plus efficacement les imperfections du système réel. Cette méthode s'avère efficace et fournit de meilleurs résultats qu'une calibration non pondérée, comme l'ont montré les expérimentations réalisées en conditions réelles dans les rues de Budapest.

La différence majeure entre l'étude de Fazekas et Gáspár et la démarche envisagée dans le cadre de la plateforme Robot Soccer Kit (RSK) réside dans le rôle attribué au réseau de neurones. Dans leur approche, une fois les segments pondérés, les constantes du modèle sont estimées en s'appuyant sur une représentation mathématique préétablie du véhicule. À l'inverse, l'objectif de la présente étude n'est pas de définir un modèle analytique du robot, mais de prédire l'ensemble de ses déplacements exclusivement à l'aide d'un réseau de neurones. Pour transposer la stratégie de Fazekas et Gáspár à ce cadre, il serait alors nécessaire d'introduire deux réseaux distincts : l'un dédié à l'identification des segments les plus pertinents, et l'autre chargé de prédire le mouvement du robot à partir des données issues de ces segments.

2.2 Augmentation de la simulation par correction d'écarts entre simulation et réalité

Plusieurs travaux récents ont exploré l'utilisation de réseaux neuronaux pour corriger les états simulés en temps réel, en s'appuyant sur des données réelles. Contrairement aux méthodes de calibration classiques, qui ajustent les paramètres d'un modèle analytique, ces approches visent à apprendre directement une correction additive des états simulés, sans modifier la structure du simulateur.

Une première catégorie de méthodes utilise des réseaux récurrents (RNN) pour prédire les écarts entre simulation et réalité. Par exemple, Golemo et al. (2018) [5] proposent d'entraîner un RNN pour estimer les différences entre le comportement simulé et réel d'un robot, puis d'intégrer cette correction dans la boucle de simulation. Cependant, ces approches sont limitées par la difficulté des RNN à capturer des dépendances temporelles longues, ainsi que par leur tendance à accumuler des erreurs sur des séquences étendues.

Plus récemment, des architectures basées sur les transformers ont été introduites pour surmonter ces limitations. Serifi et al. (2023) [14] proposent une méthode appelée State Augmentation Transformer (SAT), qui utilise un mécanisme d'attention pour corriger les états simulés des composants robotiques (actionneurs, corps rigides) de manière indépendante. Contrairement aux RNN, les transformers permettent de traiter l'ensemble de l'historique des états en parallèle, ce qui améliore significativement la précision des corrections, même sur des trajectoires complexes ou des configurations robotiques non vues pendant l'entraînement.

Les avantages du SAT sont :

- Une plus grande modularité. Chaque classe de composants (actionneurs, corps rigides) est corrigée par un SAT dédié, ce qui permet une généralisation à différents robots partageant les mêmes composants.
- Robustesse améliorée. Le mécanisme d'attention permet de capturer des dépendances temporelles longues, réduisant ainsi l'accumulation d'erreurs sur des séquences étendues.
- Prise en compte des incertitudes : Les transformers sont plus résistants aux perturbations des paramètres physiques (masse, centre de gravité, frottements), comme démontré par les expériences de l'article.

2.3 Intégration des interaction entre objet en simulation

Une problématique supplémentaire dans le cadre du projet RSK est la gestion des collision robot-robot et robot-balle. En effet un simulateur physique peut sans problème reproduire une version simplifié de ces interactions mais dans le cas d'un simulateur appris par segments, la compréhension pour le modèle du fonctionnement des collisions est longue et limité.

Une solution à ce problème est proposée par Battaglia et al. (2016) dans "Interaction Networks for Learning about Objects, Relations and Physics" [3]. Contrairement aux approches précédentes, qui traitent les systèmes comme des ensembles de données brutes ou séquentielles, les *Interaction Networks* (IN) décomposent explicitement les systèmes en objets et relations, et raisonnent sur leurs interactions de manière généralisable.

Le principe de fonctionnement de l'*Interaction Networks* est le suivant. Les IN prennent en entrée un graphe avec pour noeuds les objets et pour arêtes les relations entre les objets, puis appliquent deux types de modèles neuronaux :

- Un modèle relationnel : Prédit les effets des interactions entre paires d'objets (forces de collision, attractions gravitationnelles).
- Un modèle objet : Met à jour l'état de chaque objet en fonction des effets des interactions et des dynamiques externes.

Ce modèle repose donc sur deux réseaux de neurones fonctionnant en parallèle et échangeant des informations. En pratique, le modèle objet fait appel au modèle relationnel pour résoudre des interactions complexes, comme les collisions.

Les expériences présentées dans l'article démontrent des résultats significativement supérieurs aux modèles de base (MLP) en termes de précision et de généralisation. Par exemple, pour la prédiction des vitesses dans le domaine des cordes, l'erreur quadratique moyenne des IN est dix fois inférieure à celle des MLP classiques.

Une différence majeure avec l'approche de simulation des déplacements des robots RSK réside dans le fait que l'IN analyse la scène dans sa globalité pour déterminer quels objets peuvent interagir entre eux. En revanche, dans les modèles de déplacement classiques,

l'objectif est de découper les trajectoires du robot en segments courts afin d'améliorer la généralisation et d'éviter que le modèle n'apprenne les imperfections spécifiques du terrain pendant l'entraînement. Ainsi, il serait pertinent de combiner ces deux approches : une vision globale pour les interactions et une vision locale pour les déplacements.

3 Apprentissage par renforcement et Sim-to-real pour le robot soccer

Dans ce chapitre nous présenterons le *Reinforcement Learning*, ses applications et comment il peut être utilisé pour réduire l'écart sim to real.

3.1 Apprentissage par renforcement en robotique

Avant de détailler les grandes familles d'approches en *Reinforcement Learning* appliqué aux robots, il est nécessaire d'en introduire les principes fondamentaux et les motivations de son utilisation en robotique.

Introduction générale au RL en robotique

L'apprentissage par renforcement (Reinforcement Learning, RL) constitue aujourd'hui une méthode incontournable pour doter les robots de comportements adaptatifs et autonomes. Le RL se distingue des approches classiques de contrôle, dans lesquelles l'ingénieur doit spécifier explicitement la stratégie à appliquer, puisqu'il est basé sur le principe d'apprentissage par essai-erreur : En interagissant avec son environnement, un agent reçoit une récompense évaluant sa performance, puis ajuste progressivement sa politique afin de maximiser cette récompense cumulative. Une telle approche basée sur l'optimisation d'un comportement plutôt que sur la résolution mathématique du contrôle d'un robot, est particulièrement adaptée aux problématiques rencontrées en robotique.

Kober *et al.* (2013) montrent dans leur article -*Reinforcement Learning in Robotics : A Survey*[6]-, que de nombreux problèmes de robotique peuvent être formulés simplement comme des problèmes de RL. De plus, auteurs rappellent qu'un robot peut apprendre un

comportement en se basant sur des données issue de ses propres essais sans nécessiter une modélisation complète du système. Ceci qui constitue un réel avantage lorsque les équations de la dynamique du robot sont difficiles ou impossibles à établir. La figure 3.1, tirée de l'article de Kober *et al.*, illustre la capacité du RL à générer des comportements efficaces sur des robots très différents; Le robot Obelix a appris à pousser des objets grâce à une approche basée sur une fonction de valeur et l'hélicoptère autonome de Carnegie Mellon a exploité une approche de recherche de politique pour apprendre un contrôleur de vol.

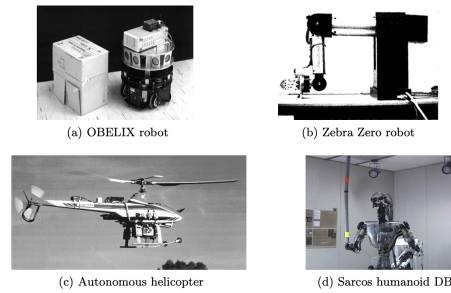


FIGURE 3.1 – Tirée de Kober *et al.* (2013) [6]. (a) Le robot Obelix est un robot roulant ayant appris à pousser des objets. (b) Le bras robotique Zebra Zero a appris à mettre une cheville dans un trou. (c) Un hélicoptère a appris un contrôleur de vol. (d) Le robot humanoïde DB de l'entreprise Sarcos utilise des modèles prédictifs pour équilibrer une perche.

Ces expériences montrent que le RL permet d'aborder de nombreux problèmes de robotique.

En résumé, le RL constitue un cadre pour apprendre des stratégies efficaces dans des environnements réels en s'appuyant sur des retours d'expérience plutôt que sur une modélisation exhaustive du système. Il s'agit ainsi d'une voie prometteuse dans laquelle différents algorithmes ont fait leur preuves.

Grandes familles d'approches en RL robotique

Dans la littérature robotique, de grandes familles de méthodes d'apprentissage par renforcement émergent : les approches *value-based* et *policy search* seront détaillées ici. Pour mieux comprendre ces méthodes il est utile d'en présenter le fonctionnement, car chacune repose sur des principes différents.

Dans cette section, les notations suivantes seront utilisées :

- Etat $s \in S$: Une description complète de la situation de l'agent dans son environnement.
- Action $a \in A$: Une commande choisie par l'agent qui modifie son état.
- Politique $\pi(a|s)$: Une distribution de probabilité sur les actions conditionnée par l'état. $\pi(a|s) = \mathbb{P}(A_t = a | S_t = s)$. C'est ce que cherche à optimiser le RL.
- Fonction de Transition $T(s, a, s')$: Donne la probabilité que l'agent arrive dans l'état s' depuis l'état s en faisant l'action a . $T(s, a, s') = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a)$. Elle définit l'environnement dans lequel évolue l'agent.
- Récompense $R(s, a)$: Nombre réel donné après une transition qui mesure la performance immédiate. C'est ce qui motive l'agent à aller dans les états voulus.

Méthodes *value-based*

Les approches basées sur la valeur *value-based* visent à apprendre une fonction qui permet d'estimer la valeur d'un état ou d'une action. Dans leur revue, Kober *et al.* [6], évoquent deux principales fonctions de valeur :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s \right] \quad (3.1)$$

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s, A_0 = a \right] \quad (3.2)$$

La fonction de valeur $V^\pi(s)$ (3.1) est l'espérance de récompenses en suivant la politique π en partant de l'état s . Le facteur $\gamma^t \in [0; 1[$ pondère l'importance des récompenses futures dans l'évaluation de s . Si γ est proche de 1, le futur compte presque autant que le présent, si γ est proche de 0 seule les récompenses immédiates comptent. La fonction de valeur d'état $Q^\pi(s, a)$ (3.2), elle, évalue une action a en étant dans un état s grâce à l'espérance des récompenses futures si l'agent choisit l'action s .

Un MDP (*Markov Decision Process*), est la description mathématique du problème que le robot doit résoudre : Optimiser les fonctions $V^\pi(s)$ ou $Q^\pi(s, a)$. Un MDP est défini par la donnée de $S, A, T(s, a, s'), R(s, a), \gamma$. Les fonctions de valeur $V^*(s)$ et $Q^*(s, a)$ sont les fonctions optimales vers lesquelles doit tendre l'apprentissage de l'agent. Dans un MDP, $V^*(s)$ et $Q^*(s, a)$ optimales sont exactement les solutions des équations de Bellman (3.3, 3.4).

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} T(s, a, s') V^*(s') \right] \quad (3.3)$$

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') \max_{a'} Q^*(s', a') \quad (3.4)$$

L'équation de Bellman pour la fonction de valeur d'état (3.3) indique qu'à l'optimum, la valeur d'un état est égale à au max de somme de la récompense immédiate disponible et à la valeur future des états dans lesquels cet état mène. Celle pour la fonction de valeur d'action (3.4) indique que la valeur d'une action est la récompense immédiate que donne cette action plus la valeur optimale des actions disponibles ensuite.

Les algorithmes de RL *value-based* sont fondés sur l'équation de Bellman : ils cherchent à apprendre une fonction V ou Q en minimisant l'écart entre leur estimation actuelle et la cible définie par Bellman. Les différents algorithmes correspondent simplement à des manières différentes d'approximer cette équation. Les auteurs identifient des algorithmes principaux :

- **TEMPORAL DIFFERENCE (TD)** : TD estime $V(s)$ en corrigeant la valeur de l'état à partir de la récompense immédiate et de la valeur estimée des états suivants.

$$V(s) \leftarrow V(s) + \alpha(R(s, a) + \gamma V(s') - V(s))$$

- **SARSA** : Cet algorithme met à jour $Q(s, a)$ à chaque action de l'agent en utilisant l'action qu'a choisie sa politique.

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s, a) + \gamma Q(s', a') - Q(s, a))$$

En résumé, l'agent apprend la valeur des choix qu'il a réellement fait. On dit que c'est une méthode *on-policy*

- **Q-LEARNING** : Pour optimiser $Q(s, a)$ la meilleure action possible est utilisée comme cible, pas l'action réellement choisie. Cette méthode est dite *off-policy*. Q-Learning est souvent plus efficace en apprentissage.

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

Dans la communauté robotique, la fonction $Q(s, a)$ est plus utilisée que $V(s)$; $V(s)$ indique seulement à l'agent si un état est bon ou mauvais, mais il ne lui indique pas quelle action il doit choisir. À l'inverse la fonction $Q(s, a)$ donne explicitement l'information de la qualité de chaque action dans un état donné, ce qui permet de faire émerger une politique en choisissant l'action maximisant Q . Un algorithme basé sur $V(s)$ doit déjà connaître la politique π . Ces différences expliquent pourquoi la majorité des algorithmes *value-based* cherchent à optimiser $Q(s, a)$.

En pratique, en robotique, les espaces S et A sont souvent continus et de grande dimension, ce qui rend l'approximation de la fonction $Q(s, a)$ difficile. De plus, le bruit et les incertitudes des capteurs et actionneurs compliquent la stabilisation des mises à jour issues de l'équation de Bellman. C'est pourquoi, dans la pratique ce sont surtout les approches *policy search* ou *model-based* qui sont privilégiées en robotique.

Méthodes *policy search*

Les méthodes *policy search* (ou *policy-based*) s'opposent aux approches *value-based* en ce qu'elles cherchent à optimiser directement la politique, plutôt que de passer par une fonction de valeur pour en déduire la meilleure action. Comme le rappellent Kober *et al.* [6], ces méthodes sont particulièrement adaptées à la robotique, car elles manipulent naturellement des espaces d'action continus, une caractéristique courante dans le contrôle de robots physiques.

Dans les approches *policy search*, la politique est représentée par un modèle paramétré par un vecteur de paramètres θ :

$$\pi_\theta(a|s)$$

L'objectif de l'apprentissage devient alors d'ajuster le vecteur θ afin de maximiser la performance globale de l'agent notée $\eta(\theta)$ définie comme l'espérance de récompenses cumulées en suivant la politique $\pi_\theta(a|s)$:

$$\eta(\theta) = \mathbb{E}_{\pi_\theta} [\sum_{t=0}^{\infty} \gamma^t R_t]$$

Le problème de contrôle robotique devient donc un problème d'optimisation des paramètres de θ .

Kober *et al.* [6] indiquent que ces méthodes sont particulièrement adaptées à la robotique, notamment parce qu'elles traitent naturellement les espaces d'action continus, courants dans ce domaine.

A. Méthodes *policy gradient* classiques

L'un des résultats majeurs pour ces méthodes est le *Policy Gradient Theorem* (3.5).

$$\nabla_\theta \eta(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) Q^{\pi_\theta}(s, a)] \quad (3.5)$$

Ce théorème montre qu'il est possible de mettre à jour la politique $\pi_\theta(s, a)$ sans avoir à estimer la fonction $Q(s, a)$ sur tout les espaces A et S , seule une approximation du retour associé aux actions réellement échantillonnées est nécessaire. Cela rend les méthodes à gradient de politique particulièrement attrayantes pour la robotique, où les espaces d'état et d'action sont continus.

Les algorithmes appliquant la méthode de *policy gradient* ne seront pas détaillées ici, notre intérêt se porte plutôt sur les algorithmes **TRPO** et **PPO**.

B. *Trust Policy Region Optimization* (TRPO)

Pour rappel, on a $\pi_\theta(a | s)$ une politique, c'est à dire un réseau de neurones qui pour un état $s \in S$ donne une action $a \in A$. On cherche à optimiser θ , les poids du réseau, pour augmenter la probabilité de faire des actions qui mènent à l'objectif du robot. Cependant, les méthodes *policy gradient* classiques manquent de stabilité, à l'apprentissage, la politique apprise peut être irrégulière. C'est précisément le problème que résout Schulman en introduisant l'algorithme de *Trust Policy Region Optimization* (TRPO) dans son article [13].

Pour assurer une stabilité de la politique $\pi_\theta(a | s)$ au fil de ses mises à jour, TRPO n'applique pas directement le gradient de politique : l'algorithme repose sur une approximation locale de la performance autour des paramètres actuels θ_{old} . Schulman introduit pour cela une fonction objectif dite *surrogate*, notée :

$$L_{\theta_{\text{old}}}(\theta)$$

Elle actualise la nouvelle politique autour des paramètres actuels θ_{old} en comparant ses performances de nouvelle politique avec l'ancienne et est définie par l'équation (3.6) :

$$L_{\theta_{\text{old}}}(\theta) = \mathbb{E}_{s, a \sim \pi_{\theta_{\text{old}}}} \left[\frac{\pi_\theta(a | s)}{\pi_{\theta_{\text{old}}}(a | s)} A^{\pi_{\theta_{\text{old}}}}(s, a) \right] \quad (3.6)$$

Dans cette expression, $\frac{\pi_\theta(a | s)}{\pi_{\theta_{\text{old}}}(a | s)}$, est la rapport d'importance (*sampling ratio*) qui indique si la nouvelle politique augmente la probabilité d'une action (ratio > 1) ou la réduit (< 1).

Le terme $A^{\pi_{\text{old}}}(s, a)$ se nomme avantage, il mesure à quel point l'action est meilleure ou pire que la moyenne actions à un état donné, défini dans l'article de Kober par [6] 3.7 :

$$A(s, a) = Q(s, a) - V(s) \quad (3.7)$$

L'algorithme fait augmenter la probabilité des actions dont $A > 0$. TRPO cherche à maximiser $L_{\theta_{\text{old}}}(\theta)$ en se basant sur tous les θ déjà visités.

Afin d'éviter que la politique ne change trop à chacune de ses mises à jour, TRPO cherche π_{new} parmi π contenus dans une certaine région autour de π_{old} . L'algorithme s'assure de rester dans une telle région en mesurant une distance entre π_{new} et π_{old} appelée *KL-Divergence*, qui ne doit pas dépasser une certaine valeur δ (3.8).

$$\mathbf{KL}(\pi_{\text{new}} || \pi_{\text{old}}) < \delta \quad (3.8)$$

Cette contrainte, qui impose que π_{new} , doit être proche de π_{old} fait que tant que la politique suivante reste dans la zone de confiance (*Trust Region*), l'approximation locale $L_{\theta_{\text{old}}}(\theta)$ est fiable, et une amélioration de la fonction *surrogate* se traduit par une amélioration de la politique $\pi_{\theta}(a | s)$. Ce qui donne le problème d'optimisation de TRPO (3.9).

$$\begin{aligned} \max_{\theta} \quad & L_{\theta_{\text{old}}}(\theta) \\ \text{s.c.} \quad & \mathbf{KL}(\pi_{\theta} || \pi_{\theta_{\text{old}}}) < \delta \end{aligned} \quad (3.9)$$

C'est grâce à ce compromis entre amélioration de la politique et mises à jour locales, que TRPO assure la mise à jour stable et monotone de la politique.

Malgré sa stabilité remarquable, TRPO présente des limites. Notamment son coût calculatoire : pour respecter la contrainte de divergence KL, l'algorithme doit résoudre un problème d'optimisation sous contrainte en utilisant le gradient conjugué ainsi qu'une approximation de la matrice de Fisher. Ces opérations sont lourdes et peu compatibles avec des réseaux neuronaux de grande taille.

C. Proximal Policy Optimization

Pour surmonter ces limitations, Schulman *et al.* introduisent ensuite l'algorithme *Proximal Policy Optimization* (PPO) [12]. PPO reprend l'idée centrale de TRPO : empêcher la politique mise à jour de trop s'éloigner de la précédente, mais remplace la contrainte explicite de KL par une pénalisation plus simple basée sur un objectif *clippé*.

PPO utilise comme point de départ le même ratio d'importance que TRPO (3.10)

$$r_{\theta}(s, a) = \frac{\pi_{\theta}(a | s)}{\pi_{\theta_{\text{old}}}(a | s)} \quad (3.10)$$

qui indique dans quelle mesure la probabilité d’une action augmente ou diminue sous la nouvelle politique. Au lieu de maximiser la fonction $L_{\theta_{\text{old}}}(\theta)$, PPO introduit un objectif *clippé* qui limite les variations excessives de ce ratio :

$$L^{\text{CLIP}}(\theta) = \mathbb{E}[\min(r_{\theta}(s, a) A(s, a), \text{clip}(r_{\theta}(s, a), 1 - \epsilon, 1 + \epsilon) A(s, a))]. \quad (3.11)$$

Le *Clipping* signifie forcer le ratio à rester dans un intervalle $[1 - \epsilon; 1 + \epsilon]$. Dans cette expression (3.11), le terme $\text{clip}(r_{\theta}, 1 - \epsilon, 1 + \epsilon)$ force le ratio à rester dans cet intervalle. Ainsi, si la nouvelle politique tend à augmenter trop fortement la probabilité d’une action ayant un avantage positif, le terme clippé annule cette incitation excessive, il en va de même pour les diminutions trop brutales qui sont empêchées lorsque $A(s, a)$ est négatif. L’objectif prend la valeur la plus pessimiste entre la version non *clippée* et la version *clippée*, ce qui garantit que l’amélioration estimée reste prudente et ne repose pas sur des changements brusques de politique.

Cette approche permet d’obtenir un comportement similaire à TRPO, mais sans avoir à résoudre un problème d’optimisation sous contrainte. PPO est donc un algorithme beaucoup plus léger et généralement plus efficace en pratique tout en conservant une stabilité comparable à celle de TRPO. Pour ces raisons, il est aujourd’hui l’une des méthodes de référence en apprentissage par renforcement pour les systèmes robotiques.

Néanmoins, passer de ces principes d’apprentissage à leur déploiement sur un robot réel révèle un ensemble de défis spécifiques au RL, qu’il convient de présenter pour comprendre les enjeux du *sim-to-real*.

Limites du RL standard et intérêt de réduire le *gap sim-to-real*

Si les méthodes de RL permettent de résoudre des problèmes complexes de contrôle, leur application directe sur des robots physiques présente des limitations. La principale difficulté réside dans le coût et la rareté des interactions réelles. Riedmiller, Gabel, Hafner et Lange (2009) [11] montrent par exemple que pour des tâches simples de robot soccer, les essais sur robot sont lents, bruyés et soumis à des défaillances mécaniques, rendant l’apprentissage par exploration directe difficilement réalisable à grande échelle.

Zhu *et al.* (2021) [17] identifie trois obstacles récurrents au RL réel :

1. Le coût temporel des transitions : les robots doivent exécuter physiquement chaque action.
2. La difficulté à collecter de grandes quantités de données en raison de l’usure matérielle.
3. La sensibilité du RL au bruit des capteurs et aux phénomènes de dynamique non modélisés (frottements, flexibilité, jeu mécanique).

Ces facteurs limitent la quantité et la qualité des trajectoires disponibles pour l’apprentissage, alors que les algorithmes modernes, comme ceux de *policy gradient* requièrent souvent plusieurs millions d’échantillons pour converger.

Face à ces contraintes, l’usage de simulateurs devient indispensable. Ils permettent de générer un grand volume de données sans risque matériel, de contrôler précisément

les conditions d'apprentissage et de répéter les expériences sans coût supplémentaire. Cependant, comme le souligne Zhu *et al.* [17], les simulateurs ne reproduisent jamais parfaitement la réalité : cette différence entre comportement en simulation et comportement réel du robot physique constitue le *sim-to-real gap*, dont la réduction est aujourd'hui un enjeu central pour l'application du RL à la robotique.

Zhu *et al.* rappellent que les algorithmes modernes de RL, en particulier en contrôle continu, nécessitent typiquement *des millions de pas d'entraînement* pour converger vers une politique performante [17]. Un tel volume d'expériences est irréalisable sur un robot physique pour des raisons d'usure, de contraintes mécaniques et de temps. Pour cette raison, la simulation joue un rôle indispensable dans le RL appliqué à la robotique.

Comme le soulignent Zhu *et al.*, s'appuyer massivement sur la simulation introduit un défi majeur : les simulateurs, même avancés, ne reproduisent jamais parfaitement les dynamiques du monde réel. Ils présentent des écarts au niveau :

- Des propriétés physiques du robot (inerties, distribution de masse).
- Des propriétés physiques de l'environnement, dans notre cas, des irrégularités dans le feutre du terrain de RSK.
- Des actionneurs (retards, saturation, pertes, non-linéarités).
- du contact sol-robot (frottements, adhérence, déformations).
- Des capteurs (bruit, latence, biais).
- Des modèles de la scène et des obstacles.

Ces écarts, documentés dans la section II de l'article de Zhu [17], provoquent ce que l'on appelle le *sim-to-real gap* (écart simulateur - réalité). C'est à dire qu'une politique optimale en simulation peut se révéler inefficace, voire dangereuse lorsqu'elle est déployée sur le robot réel.

En résumé, la simulation est indispensable pour collecter suffisamment de données d'apprentissage mais elle est à la fois insuffisante pour garantir la transférabilité des politiques apprises. Réduire l'écart sim-to-real devient donc une condition essentielle pour rendre le RL applicable sur des robots physiques. En effet le RL sur simulation peut être appliqué à des systèmes similaires au *Robot Soccer Kit* (RSK).

3.2 RL pour le *Robot Soccer*

Le domaine des robots jouant au *football* (*robot soccer*) constitue un terrain d'expérimentation privilégié pour l'apprentissage par renforcement. Dès 2009, Riedmiller *et al.* [11], ont montré que le RL permettait d'apprendre des comportements complexes tels que le dribble, le déplacement coordonné ou le tir au but grâce à des méthodes de batch RL adaptées aux environnements continus et bruités rencontrés lors de la RoboCup. Leur conclusion principale demeure d'actualité : les tâches qui constituent le *Robot Soccer* ne peuvent être entièrement programmées manuellement, et le RL fournit un moyen efficace d'apprendre directement sur les données de jeu, en s'adaptant aux dynamiques réelles du robot.

Les travaux récents de Shameer Ahamed *et al.* (2025) [15] confirment cette dyna-

mique. Ils montrent qu’une politique PPO peut apprendre à tirer des penalties ou des coups francs avec un robot capable de modifier sa géométrie pour optimiser la trajectoire du ballon. Leur système, entraîné sous *Unity*, atteint plus de 80% de réussite en penalty, illustrant la capacité du RL à exploiter des robots aux cinématiques non triviales, et à gérer un espace d’actions continu.

Tirumala *et al.* (2024)[16] optent pour une approche innovante de l’apprentissage du robot soccer. Les auteurs entraînent leurs agents uniquement à partir d’images issues de la caméra embarquée, sans capteurs externes ni informations privilégiées sur l’état du robot. Leur approche s’appuie sur un environnement simulé très réaliste, conçu pour refléter le comportement du robot dans le monde réel.

Ils mettent en évidence plusieurs phénomènes essentiels pour comprendre l’intérêt du RL dans le robot soccer :

- des comportements de perception apparaissent spontanément, comme la recherche du ballon.
- l’agent apprend simultanément des compétences de bas niveau (équilibre, marche rapide) et des comportements plus stratégiques (interception, tir).
- la robustesse de la politique dépend fortement d’un volume important de données simulées.

Tirées l’article de Tirumala, la figure 3.2 montre que l’environnement simulé est conçu pour correspondre de manière précise au terrain réel et la figure 3.3 illustre que des comportements complexes tels que l’interception ou la défense émergent au cours de l’apprentissage.

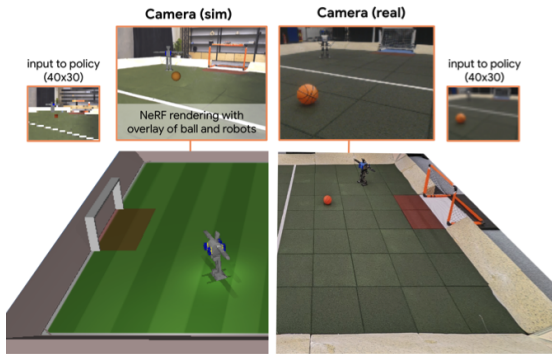


Figure 3.2 – La Figure 1 de l’article de Tirumala et al. présente l’environnement d’entraînement. Les agents sont appris entièrement en simulation, dans un environnement aligné avec celui du monde réel afin de permettre un transfert direct sans réentraînement. En simulation, la vue de la caméra est générée à l’aide d’un rendu NeRF du terrain et de l’arrière-plan statiques, auquel on ajoute les objets dynamiques.

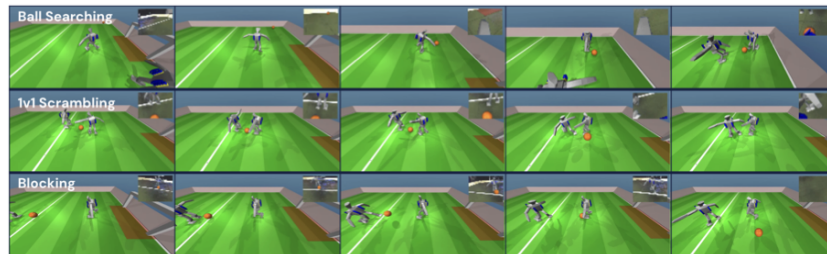


FIGURE 3.3 – La Figure 3 de l’article de Tirumala et al. illustre plusieurs comportements émergents appris par l’agent.

En résumé, l'ensemble de ces travaux montre que :

- Le *robot soccer* constitue un banc d'essai naturel pour les méthodes de RL.
- Le RL permet d'acquérir des compétences impossibles à programmer explicitement.
- La simulation est indispensable pour collecter les données nécessaires à l'apprentissage.
- Mais le transfert au réel demeure difficile, ce qui conduit à l'étude des techniques de réduction de l'écart *sim-to-real*, discutées dans la section suivante.

Ces constats motivent directement l'usage du RL pour le *Robot Soccer Kit* : son architecture simple et son environnement contrôlé en font un candidat idéal pour expérimenter les techniques modernes de RL et pour étudier les défis du *sim-to-real* à une échelle maîtrisée.

3.3 Techniques de réduction de l'écart *sim-to-real*

Dans leur revue, Zhu *et al.* soulignent que l'utilisation du RL en robotique repose sur un compromis entre, d'une part, le simulateur qui doit être suffisamment proche de la réalité pour que la politique apprise soit transférable au robot. Et d'autre part, le simulateur qui doit être assez rapide pour entraîner des algorithmes tels que PPO ou TRPO sur des millions de pas de temps [17]. Dans cette section, on s'intéresse aux approches qui agissent directement sur le simulateur, car ce sont elles qui motivent le développement d'un simulateur neuronal pour RSK.

Améliorer la fidélité du simulateur

Une première stratégie consiste à rendre le simulateur aussi fidèle que possible au système réel. C'est l'approche suivie, par exemple, dans les environnements *Isaac Lab* de NVIDIA, conçus pour la manipulation industrielle et les tâches d'assemblage [1] (2025). *Isaac Lab* combine une simulation physique détaillée (contacts, frottements, dynamique d'actionneurs) avec un entraînement RL à grande échelle sur des milliers d'environnements parallèles.

Par ailleurs il est introduit le concept de *domain randomization* : à chaque épisode de simulation, certains paramètres sont légèrement modifiés (frictions articulaires, poses des objets, etc.) [1]. L'objectif est de ne pas entraîner la politique sur un seul monde, mais sur une famille de mondes légèrement différents les uns des autres, afin de réduire la sensibilité aux variations du monde réel lors du déploiement sur la plateforme du robot. Zhu *et al.* signalent que cette randomisation de paramètres est devenue un outil standard pour améliorer la robustesse des politiques apprises[17].

Augmenter ou remplacer le simulateur par des modèles neuronaux

Une seconde stratégie consiste à ne plus se reposer uniquement sur un modèle analytique qui modélise les équations de la physique du monde et du robot. Mais à le compléter ou le remplacer par un modèle appris à partir de données. Dans ce cas, le simulateur n'est

plus uniquement défini par des équations physiques, mais par un réseau de neurones entraîné à prédire l'évolution de l'état du robot.

Serifi *et al.* (2023) [14] proposent par exemple de compléter un simulateur existant avec un réseau de type *transformer*. Leur réseau prend comme entrée l'état simulé des blocs élémentaires du robot (articulations, corps rigides, etc.) et prédit un état corrigé, plus proche de la réalité, en tenant compte imprécisions physiques (frottements, etc..) et des effets difficiles à modéliser. Cette correction est effectuée bloc par bloc, ce qui rend la méthode modulaire et plus facile à généraliser à d'autres robots.

Par ailleurs, les travaux de NVIDIA sur les *Neural Robot Dynamics* (NeRD) [9] (2025) vont plus loin en remplaçant directement le moteur physique par un modèle neuronal. NeRD apprend, à partir de grandes quantités de trajectoires simulées, à prédire l'état futur d'un robot articulé en contact avec son environnement. Une fois entraîné ce simulateur neuronal peut servir de simulateur pour apprendre des comportements via RL sur le robot. Les auteurs montrent que :

- Le modèle neuronal fournit des prédictions stables sur plusieurs centaines de pas de temps.
- Un seul modèle peut généraliser à différents environnements et tâches pour un même robot.
- Le modèle peut être affiné sur des données réelles, ce qui améliore la précision et réduit l'écart sim-to-real.

Ces travaux illustrent une tendance claire : plutôt que d'essayer de rendre le modèle analytique parfait, on accepte ses limites et on introduit une composante neuronale chargée de corriger les erreurs systématiques ou de capturer des phénomènes non modélisés (contacts complexes, jeux mécaniques, usure, etc.).

Comme le soulignent Zhu *et al.*, cette combinaison entre simulateurs enrichis (analytiques ou neuronaux) et RL profond constitue aujourd'hui l'une des voies les plus prometteuses pour déployer effectivement le RL sur des robots physiques [17]. Dans la section suivante, on examine comment ces principes peuvent être adaptés au cas particulier de la plateforme RSK, en construisant un simulateur neuronal capable de réduire l'écart entre le modèle analytique existant et le comportement réel des robots omnidirectionnels.

Perspectives pour le simulateur neuronal de RSK

Dans le contexte de RSK, ces techniques fournissent un cadre conceptuel clair pour notre projet. Le simulateur analytique actuel est volontairement simple et ne modélise ni les effets de glissement, ni les détails des moteurs, ni du terrain. Il est donc peu adapté à un entraînement de politiques RL déployées ensuite sur les robots physiques. Les approches d'augmentation neuronale de simulateurs [14, 1, 9] suggèrent une stratégie particulièrement adaptée à une plateforme pédagogique comme RSK :

- Conserver un modèle analytique léger, qui encode la structure cinématique du robot omnidirectionnel.
- Apprendre, à partir de données réelles de RSK, un modèle de transition neuronal

qui corrige la dynamique simulée ou prédit directement l'état futur du robot.

L'objectif n'est donc pas de répliquer les infrastructures lourdes utilisées en industrie, mais de transposer leurs principes à l'échelle d'une plateforme éducative : un simulateur plus fidèle, enrichi par un modèle appris à partir des trajectoires de RSK, afin de réduire l'écart sim-to-real et de rendre le RL réellement exploitable pour le robot soccer sur cette plateforme.

Conclusion & perspectives

Le simulateur de RSK repose sur un modèle analytique qui décrit la cinématique omnidirectionnelle du châssis et intègre une dynamique simplifiée au niveau des vitesses et des collisions. Ce modèle ne représente pas explicitement la dynamique des moteurs, les frottements continus, le glissement des roues, ni les incertitudes de localisation, et suppose une pose parfaitement connue. La littérature sur la calibration d'odométrie et la modélisation de véhicules montre qu'un tel modèle analytique, non ajusté sur des données réelles, entraîne un écart significatif entre déplacement simulé et déplacement observé.

La littérature comporte peu de travaux s'appuyant sur un simulateur entièrement appris sans base physique, ce qui suggère qu'un modèle purement neuronal pose rapidement des problèmes de stabilité, de généralisation ou d'interprétabilité. Dans le contexte du Robot Soccer Kit, le modèle actuel pourrait ainsi servir de noyau déterministe, auquel seraient ajoutées, dans un second temps, soit des procédures de calibration inspirées des travaux sur l'odométrie, soit des modules neuronaux visant à rapprocher la dynamique simulée de celle observée sur les robots physiques.

Application sur la plateforme RSK

Les solutions proposées dans la littérature pour améliorer la fidélité des simulations de mouvements sur une plate-forme robotique reposent généralement sur une architecture modulaire. Celle-ci combine plusieurs réseaux de neurones, soit indépendants, soit connectés, chacun étant dédié à une tâche spécifique : simulation des interactions physiques, modélisation des déplacements, ou encore calibration des paramètres.

L'utilisation de réseaux de neurones dédiés à une tâche nous permet donc une meilleure compréhension de la physique de la part du modèle, mais évite aussi les problèmes que nous pourrions rencontrer avec un modèle monolithique, comme la complexification de l'entraînement, une baisse de la généralisation et une perte de précision.

Bibliographie

- [1] Bridging the sim-to-real gap for industrial robotic assembly applications using NVIDIA isaac lab.
- [2] Robot soccer kit documentation.
- [3] Peter Battaglia, Razvan Pascanu, Matthew Lai, Danilo Jimenez Rezende, and Koray kavukcuoglu. Interaction networks for learning about objects, relations and physics. In *Proceedings of the 30th International Conference on Neural Information Processing Systems*, NIPS'16, pages 4509–4517. Curran Associates Inc.
- [4] Máté Fazekas and Péter Gáspár. Wheel odometry model calibration with neural network-based weighting. 134 :108631.
- [5] Florian Golemo, Adrien Ali Taiga, Aaron Courville, and Pierre-Yves Oudeyer. Sim-to-real transfer with neural-augmented robot simulation. In *Proceedings of The 2nd Conference on Robot Learning*, pages 817–828. PMLR. ISSN : 2640-3498.
- [6] Jens Kober, J Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics : A survey.
- [7] L. Lemmer, R. Heß, M. Krauß, and K. Schilling. Calibration of a car-like mobile robot with a high-precision positioning system*. 43(23) :174–179.
- [8] Kevin M. Lynch and Frank C. Park. *Modern Robotics : Mechanics, Planning, and Control*. Cambridge University Press, 1 edition.
- [9] NVIDIA. Advancing robotics development with neural dynamics in newton.
- [10] Gregoire Passault, Clement Gaspard, and Olivier Ly. Robot soccer kit : Omniwheel tracked soccer robots for education. In *2022 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*, pages 34–39. IEEE.
- [11] Martin Riedmiller, Thomas Gabel, Roland Hafner, and Sascha Lange. Reinforcement learning for robot soccer. 27(1) :55–73.
- [12] John Schulman, Prafulla Dhariwal, Filip Wolski, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms.
- [13] John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel. Trust region policy optimization.
- [14] Agon Serifi, Espen Knoop, Christian Schumacher, Naveen Kumar, Markus Gross, and Moritz Bächer. Transformer-based neural augmentation of robot simulation representations. 8(6) :3748–3755.
- [15] M. Shameer Ahamed, J. J. J. Pey, S. M. Bhagya P. Samarakoon, M. A. Viraj J. Muthugala, and Mohan Rajesh Elara. Reinforcement learning for reconfigurable robotic soccer. 13 :22314–22324.

- [16] Dhruva Tirumala, Markus Wulfmeier, Ben Moran, Sandy Huang, Jan Humplik, Guy Lever, Tuomas Haarnoja, Leonard Hasenclever, Arunkumar Byravan, Nathan Batchelor, Neil Sreendra, Kushal Patel, Marlon Gwira, Francesco Nori, Martin Riedmiller, and Nicolas Heess. Learning robot soccer from egocentric vision with deep reinforcement learning.
- [17] Wei Zhu, Xian Guo, Dai Owaki, Kyo Kutsuzawa, and Mitsuhiro Hayashibe. A survey of sim-to-real transfer techniques applied to reinforcement learning for bioinspired robots. PP :1–16.